Laporan Tugas Kecil 1

IF2211 Strategi Algoritma

Penyelesaian Permainan Queens Linkedin

Semester 2 Tahun 2025/2026

Dipersiapkan oleh:

Muhammad Jordan Ferimeison (13524047)

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

1. Desain dan Implementasi

a. Desain

Program dibuat menggunakan algoritma *brute force exhaustive search*. Program akan menerima masukkan pada file “input.txt” serta mengeluarkan hasil pada terminal dan “output.txt”. Program melakukan iterasi pada setiap kemungkinan permutasi papan secara berurut. Pada setiap iterasi, program akan menguji apakah solusi yang dihasilkan memenuhi syarat. Syarat yang perlu dipenuhi pada setiap iterasi adalah,

- Tidak ada *Queen* yang *adjacent* (berjarak 1 pada jarak euclidean)
- Terdapat maksimal 1 *Queen* pada tiap baris dan kolom
- Terdapat tepat 1 *Queen* pada setiap daerah warna

Selain itu, program harus dapat menghitung durasi dan banyaknya iterasi yang diperlukan hingga menemukan solusi. Pada program yang akan dibuat, permutasi dibuat pada papan kosong(hanya berisi ‘_’ dan ‘#’). Hal ini dilakukan agar identifikasi posisi *Queen* lebih mudah. Hasil permutasi pada papan kosong nantinya akan di-*cast* pada papan input. Selanjutnya, program baru meninjau apakah solusi tersebut memenuhi syarat.

b. Implementasi

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long


string mainString = "", alphabetKey = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
set<char> color;

ll n = 0, m = 0;

int main(){
        ifstream fin("test.txt");
        string temp;
        while(getline(fin, temp)){
                if(temp.empty()) break;
                else mainString.append(temp);
                n++;
                if (m == 0) m = temp.size();
                else if (m != temp.size()){
                        cout << "Input invalid. Panjang setiap baris harus sama." << '\n';
                        return 0;
                }
                for (int i = 0; i < m; i++){
                        int location = alphabetKey.find(temp[i]);
                        if (location == string::npos){
                                cout << "Input invalid. Karakter tidak dikenali. List karakter
valid: ";
                                for(int i=0; i<alphabetKey.size(); i++) cout << alphabetKey[i]
<< ' ';
```

```
                                cout << '\n';
                                return 0;
                        }
                        else color.insert(temp[i]);
                }
        }

        ll colorNum = color.size();
        string cmpr = "", ans = "";

        for (int i=0; i<n; i++){
                cmpr.append(i, '_');
                cmpr.append(1, '#');
                cmpr.append(m-i, '_');
        }
        bool valid = false;
        ll iter = 0;

        time_t startTime, endTime;
        time(&startTime);

    do {
                set<char> unusedColor = color;
                valid = true;
                iter++;
                for(int i=0; i<n; i++){
                        for(int j=0; j<m; j++){
                                if (cmpr[i*m+j] == '#'){
                                        if (unusedColor.find(mainString[i*m+j]) ==
unusedColor.end()){
                                                valid = false;
                                                break;
                                        }
                                        unusedColor.erase(mainString[i*m+j]);
                                        for(int k=0; k<n; k++){
                                                if (k != i && cmpr[k*m+j] == '#'){
                                                        valid = false;
                                                        break;
                                                }
                                        }
                                        for(int k=0; k<m; k++){
                                                if (k != j && cmpr[i*m+k] == '#'){
                                                        valid = false;
                                                        break;
                                                }
                                        }
                                        if (((i-1)*m + (j-1) >= 0 && cmpr[(i-1)*m + (j-1)] ==
'#') || ((i-1)*m + (j+1) >= 0 && cmpr[(i-1)*m + (j+1)] == '#')){
                                                valid = false;
                                                break;
                                        }
                                }
                        }
                        if (!valid) break;
                }

                if (!unusedColor.empty()) valid = false;

                if (iter % 1000000 == 0 || valid){
                        if (!valid) cout << "Melakukan iterasi..." << '\n';
                        else cout << "Solusi valid ditemukan!" << '\n';
                        for(int i=0; i<n; i++){
                                for(int j=0; j<m; j++){
                                        if (cmpr[i*m+j] == '#') cout << cmpr[i*m+j];
                                        else cout << mainString[i*m+j];
                                }
                                cout << '\n';
```

```
                        }
                        cout << '\n';
                        if (valid) ans.append(cmpr);
                }

        } while (next_permutation(cmpr.begin(), cmpr.end()) && !valid);

        time(&endTime);

        if (!valid) cout << "Tidak menemukan solusi valid." << '\n';


        cout << "Waktu pencarian: " << difftime(endTime, startTime) << " detik" << '\n';
        cout << "Banyak kasus yang ditinjau: " << iter << '\n';
        cout << "Menyimpan jawaban..." << '\n';
        ofstream fout("output.txt");
        for(int i=0; i<n; i++){
                for(int j=0; j<m; j++){
                        if (ans[i*m+j] == '#') fout << ans[i*m+j];
                        else fout << mainString[i*m+j];
                }
                fout << '\n';
        }
        fout << "Waktu pencarian: " << difftime(endTime, startTime) << " detik" << '\n';
        fout << "Banyak kasus yang ditinjau: " << iter << '\n';
        fin.close();
        fout.close();
}
```

Kode sumber: github.com/Nyanist/Tucil1_13524047

2. Eksperimen

AA BB Waktu pencarian: 0 detik Banyak kasus yang ditinjau: 12
Input invalid. Karakter tidak dikenali. List karakter valid: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Input invalid. Panjang setiap baris harus sama.
ABAA AAAC DCCC DDDC Waktu pencarian: 0 detik Banyak kasus yang ditinjau: 4185
A#AA AAA#

#CCC DD#D Waktu pencarian: 0 detik Banyak kasus yang ditinjau: 845
#BCDEFGHI AB#DEFGHI ABCD#FGHI ABCDEFG#I A#CDEFGHI ABC#EFGHI ABCDE#GHI ABCDEFGH# ABCDEF#HI Waktu pencarian: 5 detik Banyak kasus yang ditinjau: 2044281

3. Kesimpulan dan Evaluasi

Pada program yang telah dibuat, program perlu menjalankan permutasi sebanyak $P^{n*m}_{colorNum}$, permutasi dari banyaknya warna pada perkalian antara baris dan kolom. Pada setiap permutasi, dilakukan pengecekan ke setiap baris, kolom, dan petak yang bersebelahan. Pengecekan ini dilakukan pada N*M buah petak. Pada setiap petak Queen, diperlukan N buah pengecekan untuk menguji petak kolom, M buah pengecekan untuk menguji petak baris, dan 2 buah pengecekan untuk mengecek petak yang bersebelahan. Dengan demikian, diperlukan komputasi sebanyak $P^{n*m}_{colorNum} * ((N * M - colorNum) + (colorNum * (N + M + 2)))$ atau memiliki kompleksitas $O(N! * N * M)$. Untuk membuat program menjadi lebih sangkil, penulis memikirkan beberapa solusi alternatif. Misal, jika dipastikan terdapat N buah warna pada petak N x N, maka dapat membagi permasalahan menjadi baris/kolom/daerah warna individu. Tiap daerah nantinya akan memiliki 1 Queen dan digabung di akhir (seperti konsep *divide* pada metode *divide and conquer*). Solusi lain seperti menggunakan fungsi rekursif dan heuristik juga dapat dilakukan. Selain itu, program yang dibuat dapat menimbulkan masalah seperti *integer overflow* pada variabel iter (yang ditujukan untuk menghitung banyaknya iterasi). Solusi

sementara yang dilakukan adalah dengan memberikan tipe data long long yang lebih besar dari tipe data integer

4. Penutup

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	x	
2	Program berhasil dijalankan	x	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	x	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	x	
5	Program memiliki Graphical User Interface (GUI)		x
6	Program dapat menyimpan solusi dalam bentuk file gambar		x

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Jordan Ferimeison